Assignment - Backend Node.js

Format

This assignment has two parts:

- Part 1: Architecture Principles - written response. No code implementation required.
- Part 2: Backend Development - backend implementation required.

You can choose the presentation format for Part 1 (Notion, Figjam or the one you profer), it can include diagrams, code snippets, and justifications of the decisions when considered necessary. The backend implementation for Part 2 can be demonstrated using Swagger, Postman, automated tests, a short loom video, a simple frontend, or any other format that clearly shows the implemented behavior.

The goal is to understand how you reason through and approach complex problems, while evaluating your proposed backend architecture, data modeling, data-structure reasoning, API design, correctness, maintainability, and ability to explain tradeoffs.

Part 1 - Architecture Principles

The Scenario

You are asked to design a backend architecture for an english learning product through music. The product stores songs with synchronized lyrics, translations, and word-level learning insights that help users understand, practice, and track vocabulary, including normalized words, translations, difficulty, frequency, examples, and learning status.

Users can also open a song, read the lyrics, see translations, tap words, and understand which words they already know or still need to learn.

Song lyrics data models must allow the app to display the lyrics in order, showing line-level or word-level translations, searching words inside a song, finding word occurrences, generating vocabulary insights, calculating song difficulty for a specific user, and tracking user vocabulary state.

This part is written-only. You are not expected to implement the full lyrics system.

The Goal

Design a scalable and maintainable backend architecture that supports lyrics, translations, word insights, and user vocabulary state. Your design should clearly separate:

- Original lyric structure used for display.
- Searchable/indexable word representation used for queries.
- Translation model for line-level and word-level translations.
- User-specific vocabulary state.
- Insight generation process.

Example Input Data

The backend may receive songs in a structure similar to the following. The exact DTO is intentionally not fixed; explain what you would keep, change, normalize, or enrich.

```
{
  "songId": "song_001",
  "title": "Example Song",
  "artist": { "artistId": "artist_001", "name": "Example Artist" },
  "language": "en",
  "lyrics": [
    {
      "lineIndex": 0,
      "startTimeMs": 1200,
      "endTimeMs": 4200,
      "text": "I found a love for me",
      "translation": { "language": "es", "text": "Encontré un amor para mí" }
    }
  ]
}
```

Expected Backend Capabilities

- **Song and lyrics storage**: preserve song metadata, artist metadata, lyric line order, original text, normalized word (e.g. normalized word for “done”, can be “do”), optional timestamps, translations, and word metadata.
- **Word indexing**: explain how lyric lines become searchable word-level data, position, line reference, timing, generation, storage, updates, and query strategy.
- **User vocabulary state**: store user-specific vocabulary independently from the song catalog. This will determine if the user had already learnt a word. Possible statuses: known, learning, unknown, ignored.
- **Word occurrence query**: given a song and word, return line position, word position, original line text, normalized word, and optional timing data.
- **Line and word translation query**: return available translations for a lyric line or specific word occurrence, and explain how missing translations are represented.
- **Song insights query**: given a song and user, return total words, unique words, known/unknown words, repeated words, relevant user vocabulary status, and a difficultyScore.

For this challenge, difficultyScore means how difficult a specific song is for a specific user. It can be a number between 0 and 1, where 0 is very easy and 1 is very difficult. A simple acceptable model is unknownUniqueWords / totalUniqueWords. You can propose a richer model if it remains explainable, testable, and easy to change.

What Your Written Submission Should Cover

- Proposed data model, main entities, and relationships.
- How lyrics are imported, tokenized, normalized, and indexed.
- How translations are represented and resolved (e.g. if the user language is missing from the lyrics translations data).
- How core operations are performed and which operations are synchronous or asynchronous.
- What optimization strategies / patterns you consider beneficial for the expected capabilities.
- How errors and invalid input are handled at a high level.
- How the system could scale with more songs, users, and languages.
- Example DTOs, diagrams, or pseudo-code where helpful.
- Assumptions regarding third party services your solution requires.

Out of Scope for Part 1

- Full implementation of the system.
- Machine learning models.
- Infrastructure details such as Kubernetes, Terraform, or cloud provider setup.
- Detailed frontend behavior.

Part 2 - Backend Development

The Scenario

You are asked to implement a backend service that helps users practice vocabulary using word insights. Part 1 explains how a larger system could produce insights from songs, lyrics, translations, and user vocabulary state. Part 2 should not implement that full lyrics system.

Instead, assume that word insights already exist and build a practice layer on top of them. Users should be able to learn words by completing exercises generated from stored insight data.

The Goal

Build a backend that can import or seed word insights, track user vocabulary state, prioritize words for practice, generate exercises, store practice sessions and attempts, update user learning state, and return a user-specific insight summary.

Domain Rules

All exposed models are examples. You are expected to challenge them and tweak them if needed, as long as your decisions are documented.

Word Insights

A word insight represents a word that may be useful for a user to learn or practice. Support at least: id, word, normalizedWord, language, translations, difficulty, frequency, source, songRefs, imageRefs, and optional examples.

The translations field should support multiple target languages. imageRefs can be used for image-based exercises. songRefs should be lightweight and should not require full lyrics processing. The examples field is also optional and can provide sentence context without implementing lyric parsing.

```
{
  "id": "insight_001",
  "word": "darling",
  "normalizedWord": "darling",
  "language": "en",
  "translations": [{ "language": "es", "text": "cariño" },{ "language": "pt", "text": "querido"
}],
  "difficulty": 2,
  "frequency": 12,
  "source": "song",
  "songRefs": [{ "songId": "song_001", "title": "Example Song", "occurrences": 1 }],
  "imageRefs": [{ "id": "image_darling_001", "url": "https://example.com/image.png" }],
  "examples": [{ "text": "Darling, just dive right in", "translations": [{ "language": "es",
"text": "Cariño, simplemente lánzate" }]]
}
```

User Vocabulary - User state

User vocabulary should be stored separately from global word insights. This entity indicates the status of knowledge of a word for a user. Possible statuses are unknown, learning, known, and ignored.

The backend should also be able to update a user's vocabulary state directly or as a result of exercise attempts. Related to the attempts, the backend needs to track the correct attempt count, incorrect attempt count, and last practiced date.

Practice Prioritization

Decide which word insights are most relevant for a user to practice. A simple prioritization rule is enough. For example: prefer unknown or learning words, words from songs the user interacted with, higher-frequency words, or words answered incorrectly recently. Document the scoring or sorting rule you choose.

Practice Exercises

Generate exercises from stored word insights, not from hardcoded questions. Implement at least two exercise types, such as:

- word_meaning: choose the correct translation or meaning of a word.
- translation_match: match a word with its translation.
- reverse_translation: choose the source-language word for a translated meaning.
- word_to_image: choose the image that best represents a word.

Only generate translation exercises when the requested translationLanguage is available, and only generate image exercises when enough image data exists to build a valid question and answer set.

Exercise Attempts

When a user answers an exercise, store an attempt with userId, exerciseId, wordInsightId, word, isCorrect, answer, and createdAt. Update the user vocabulary using a simple documented rule, such as marking a word as known after two correct answers and moving it to learning after an incorrect answer (or the chosen rule that you prefer).

Required Backend Inputs and Outputs

Endpoint names, HTTP methods, and route structure are flexible. Expose equivalent capabilities and document each input and output shape.

Operation	Expected behavior
Import Word Insights	Import or upsert global word insights; summarize created, updated, skipped, or rejected records.
Get Word Insights	Return global insights with filters such as language, source, difficulty, normalized word, and pagination.
Get User Word Insights	Combine global insights with user vocabulary state, practice stats, priority score, and recommendation reason.
Update User Vocabulary	Manually update a user vocabulary status for a word insight and return the updated state.
Get User Insight Summary	Return counts by status, aggregate attempt stats, and recommended words to practice.
Create Practice Session	Create a session with generated exercises based on limit, source language, translation language, statuses, and exercise types.
Submit Exercise Attempt	Record an answer, validate correctness, and return previous and new vocabulary status when applicable.
Get Practice Session Results	Return completed and pending exercises, correct/incorrect counts, and latest vocabulary state for practiced words.

Example Dataset

You will find an example dataset in the asset included. You can use it, tweak it, or create your own dataset.

Technical Expectations

- Use Node.js, TypeScript and MongoDB.
- Define clear schemas and collection boundaries.
- Use indexes that support common queries, for example by normalizedWord, userId, language, status, and wordInsightId.
- Keep prioritization and exercise generation deterministic and easy to test.

Deliverable

Submit a repository or project folder containing:

- Backend implementation.
- Instructions to run the project.
- MongoDB setup instructions or Docker Compose if used.

You may include Swagger, a Postman collection, automated tests, a demo video, or a simple frontend if helpful. The way the backend is presented is intentionally open.

Out of Scope for Part 2

- Full lyrics import or parsing.
- Word occurrence extraction from lyrics.
- Authentication and authorization.
- Production deployment.
- Production-ready translation generation.
- Advanced recommendation algorithms.
- Machine learning.
- Payment, subscriptions, account management.
- Complex frontend implementation.